assignment_2.2_deliverable

November 1, 2026

Your assignment is to find the shortest driving distance between Oracle Park (home of the San Francisco Giants) and the Palace of Fine Arts (another one of San Francisco's gems). These two locations span a large portion of the city and are connected through various major roads. This problem mimics real-world GPS applications where we seek to minimize travel time (or distance) over a road network.

1) Convert this search problem into Python code using NetworkX and OSMnx. Use at least two search algorithms to solve the problem. Explain why you chose the algorithms you did.

2) If you had additional time or knowledge (we can only cover so much in AAI-501!), how would you improve on your solution to your search problem? That is, what other data, information, heuristic improvements, etc., would you like to leverage? (3-4 paragraphs)

[1]:
```
import networkx as nx
import matplotlib.pyplot as plt
import osmnx as ox
```

[5]:
```
o_park_address = "24 Willie Mays Plaza, San Francisco, CA 94107"
palace_address = "3601 Lyon St, San Francisco, CA 94123"

o_park_coords = ox.geocode(o_park_address)
palace_coords = ox.geocode(palace_address)

center = ((o_park_coords[0] + palace_coords[0]) / 2, (o_park_coords[1] +␣
 ↪palace_coords[1]) / 2)
dist = max(ox.distance.great_circle(o_park_coords[0], o_park_coords[1],␣
 ↪center[0], center[1]), ox.distance.great_circle(palace_coords[0],␣
 ↪palace_coords[1], center[0], center[1]))
graph = ox.graph_from_point(center_point=center, dist=dist)

weighted_graph = ox.distance.add_edge_lengths(graph)

o_park_node = ox.distance.nearest_nodes(weighted_graph, o_park_coords[1],␣
 ↪o_park_coords[0])
palace_node = ox.distance.nearest_nodes(weighted_graph, palace_coords[1],␣
 ↪palace_coords[0])

def a_star_heuristic(u, v):
    u_coords = (weighted_graph.nodes[u]['y'], weighted_graph.nodes[u]['x'])
```

```
    v_coords = (weighted_graph.nodes[v]['y'], weighted_graph.nodes[v]['x'])
    return ox.distance.great_circle(u_coords[0], u_coords[1], v_coords[0],␣
 ↪v_coords[1])

a_star_path = nx.astar_path(weighted_graph, o_park_node, palace_node,␣
 ↪heuristic=a_star_heuristic, weight='length')
a_star_length = nx.astar_path_length(weighted_graph, o_park_node, palace_node,␣
 ↪heuristic=a_star_heuristic, weight='length')

print(o_park_node)
print("A* path:", a_star_path)
print("A* path length:", a_star_length)
```

```
6329652599
A* path: [6329652599, 11590445288, 11590445289, 6318866712, 6318866711,
6329652898, 7209260204, 1270672015, 7209260203, 7228380563, 7209260199,
7209260196, 7209260200, 7228376737, 9234607022, 10875001004, 9234607037,
65299118, 448099743, 65300056, 7209260188, 65290217, 65300059, 5839244979,
5839244981, 7228376742, 65300069, 5429899862, 5429899865, 7209252645, 65300073,
7209252647, 6896819881, 5429899847, 65300076, 11897570033, 4547464638, 65300078,
6375094234, 10211440809, 65300080, 10696024400, 10696024402, 5554083626,
65300085, 5554083622, 65300091, 65300093, 12452571768, 7228376766, 65300097,
7228376767, 65300105, 65300107, 1723738952, 1723738990, 1419794119, 65334111,
6339001616, 65348742, 12145090989, 65314146, 10740557525, 65302739, 10178189137,
65303533, 10178189136, 65360482, 12280846486, 65319923, 12280846484,
12280846480, 65352429, 12280846482, 11566575703, 1206893700, 7231084727,
65295289, 7231084728, 12280846505, 65307363, 10740803906, 65356185, 4518026430,
65336953, 10002543573, 10002543545, 65373233, 10002543526, 4177198638,
10002543533, 4493528407, 65374771, 4518026431, 7233567557, 65337385, 7233567559,
7231121749, 7231121753, 7231121754, 7231121748, 7231121759, 7231323907,
7231121743, 6514737765, 416351394, 7231700360, 7231121739, 6514737761,
6514737760, 7231121741, 6378899302, 330665612, 12954382074, 4019577201,
6378899306, 7231121733, 4011019098, 7231121730, 7231323894, 7231176139,
7231176134, 7231176142, 13830563425, 7231323840, 7231285581, 7231323841,
1582797136, 612765105, 1582797140, 1582797147, 1582797152, 7231176132,
4438409335, 7231176128, 13323204410, 13323162423, 1582797178, 1582797185,
13323162427, 7231176124, 1582797193, 1582797202, 7231323830, 7231176120,
7231176116, 7231176123, 7231323826, 7231323825, 12950555096, 9235029400,
7231430157, 4527874994, 7231443634, 65296822, 7231443632, 7231443643, 65296824,
7231443644, 1594227872, 65296827, 7231443649, 65290876, 7895860238, 7231449992,
65296833, 7231449990, 4728918107, 260336116, 65296837, 4728918103, 1304953094,
4728917986, 65296839, 4728917985, 65296846, 2972640105, 65296848, 2972640103,
6816344134, 542900782, 65296849, 2972640086, 5853135789, 10748021637, 65292804,
10748071431, 12952717259, 65296850, 12952717255, 4025675327, 4009044899,
4009044924, 65296852, 258956662, 65345875, 2980617845, 65345881, 4009068077,
12288476784, 65297322, 12288476782, 12913487197, 4009069589, 12913487190,
12952532779, 4008988381, 65353051, 12952573222, 12952573220, 4009069604,
```

```
4009068070, 8465187968, 12952716159, 12952716154, 12952716153, 4409247021,
12952716150, 12952716146, 12952716144, 4409247013, 12952716142, 12952716141,
12952716138, 4409247003, 12952716160, 4719866630, 12750138270, 12750138272]
A* path length: 6863.391406964465
```

[6]:
```
dijkstra_path = nx.dijkstra_path(weighted_graph, o_park_node, palace_node,␣
 ↪weight='length')
dijkstra_length = nx.dijkstra_path_length(weighted_graph, o_park_node,␣
 ↪palace_node, weight='length')
print("Dijkstra path:", dijkstra_path)
print("Dijkstra path length:", dijkstra_length)
```

```
Dijkstra path: [6329652599, 11590445288, 11590445289, 6318866712, 6318866711,
6329652898, 7209260204, 1270672015, 7209260203, 7228380563, 7209260199,
7209260196, 7209260200, 7228376737, 9234607022, 10875001004, 9234607037,
65299118, 448099743, 65300056, 7209260188, 65290217, 65300059, 5839244979,
5839244981, 7228376742, 65300069, 5429899862, 5429899865, 7209252645, 65300073,
7209252647, 6896819881, 5429899847, 65300076, 11897570033, 4547464638, 65300078,
6375094234, 10211440809, 65300080, 10696024400, 10696024402, 5554083626,
65300085, 5554083622, 65300091, 65300093, 12452571768, 7228376766, 65300097,
7228376767, 65300105, 65300107, 1723738952, 1723738990, 1419794119, 65334111,
6339001616, 65348742, 12145090989, 65314146, 10740557525, 65302739, 10178189137,
65303533, 10178189136, 65360482, 12280846486, 65319923, 12280846484,
12280846480, 65352429, 12280846482, 11566575703, 1206893700, 7231084727,
65295289, 7231084728, 12280846505, 65307363, 10740803906, 65356185, 4518026430,
65336953, 10002543573, 10002543545, 65373233, 10002543526, 4177198638,
10002543533, 4493528407, 65374771, 4518026431, 7233567557, 65337385, 7233567559,
7231121749, 7231121753, 7231121754, 7231121748, 7231121759, 7231323907,
7231121743, 6514737765, 416351394, 7231700360, 7231121739, 6514737761,
6514737760, 7231121741, 6378899302, 330665612, 12954382074, 4019577201,
6378899306, 7231121733, 4011019098, 7231121730, 7231323894, 7231176139,
7231176134, 7231176142, 13830563425, 7231323840, 7231285581, 7231323841,
1582797136, 612765105, 1582797140, 1582797147, 1582797152, 7231176132,
4438409335, 7231176128, 13323204410, 13323162423, 1582797178, 1582797185,
13323162427, 7231176124, 1582797193, 1582797202, 7231323830, 7231176120,
7231176116, 7231176123, 7231323826, 7231323825, 12950555096, 9235029400,
7231430157, 4527874994, 7231443634, 65296822, 7231443632, 7231443643, 65296824,
7231443644, 1594227872, 65296827, 7231443649, 65290876, 7895860238, 7231449992,
65296833, 7231449990, 4728918107, 260336116, 65296837, 4728918103, 1304953094,
4728917986, 65296839, 4728917985, 65296846, 2972640105, 65296848, 2972640103,
6816344134, 542900782, 65296849, 2972640086, 5853135789, 10748021637, 65292804,
10748071431, 12952717259, 65296850, 12952717255, 4025675327, 4009044899,
4009044924, 65296852, 258956662, 65345875, 2980617845, 65345881, 4009068077,
12288476784, 65297322, 12288476782, 12913487197, 4009069589, 12913487190,
12952532779, 4008988381, 65353051, 12952573222, 12952573220, 4009069604,
4009068070, 8465187968, 12952716159, 12952716154, 12952716153, 4409247021,
12952716150, 12952716146, 12952716144, 4409247013, 12952716142, 12952716141,
12952716138, 4409247003, 12952716160, 4719866630, 12750138270, 12750138272]
```

```
Dijkstra path length: 6863.391406964465
```

I chose A* because the goal was known and on a weighted graph by length. It only makes sense for the graph to be weighted since it's asking for shortest driving distance. Since it's asking for shortest driving distance, both A* and dijkstras pathing algorithms give the shortest

If I had more time I would like to know speed limits so we could calculate the fastest route too and then see what the difference would be if the same algorithms used time as the weight instead of distance.

I would also like to try some of the other algorithms mentioned in the lectures but networkx doesn't have them so I would have to write them myself and right now it would take me about an hour to learn and write it.

Lastly, I would like to try to include other details like traffic and elevation and see how that affects the outcomes.

Disclaimer: AI helped in grammar checks and debugging of code